

Chapter 20. Network Policies

The uniqueness of the IP address assigned to a Pod is maintained across all nodes and namespaces. This is accomplished by allocating a dedicated subnet to each registered node during its creation. The Container Network Interface (CNI) plug-in handles the leasing of IP addresses from the assigned subnet when a new Pod is created on a node. Consequently, Pods on a node can seamlessly communicate with all other Pods running on any node within the cluster.

By default, Kubernetes allows unrestricted Pod-to-Pod communication across all namespaces, which poses a significant security risk, as a compromised Pod in one namespace could potentially access sensitive services in another namespace.

Network policies in Kubernetes function similarly to firewall rules, specifically designed for governing Pod-to-Pod communication. These policies include rules specifying the direction of network traffic (ingress and/or egress) for one or multiple Pods within a namespace or across different namespaces. Additionally, these rules define the targeted ports for communication. This fine-grained control enhances security and governs the flow of traffic within the Kubernetes cluster.

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objective:

- Define and enforce Network Policies
-

Working with Network Policies

Within a Kubernetes cluster, any Pod can talk to any other Pod without restrictions using its [IP address or DNS name](#), even across namespaces. Not only does unrestricted inter-Pod communication pose a potential security risk, it also makes it harder to understand the mental communication model of your architecture. A network policy defines the rules that control traffic from and to a Pod, as illustrated in [Figure 20-1](#).

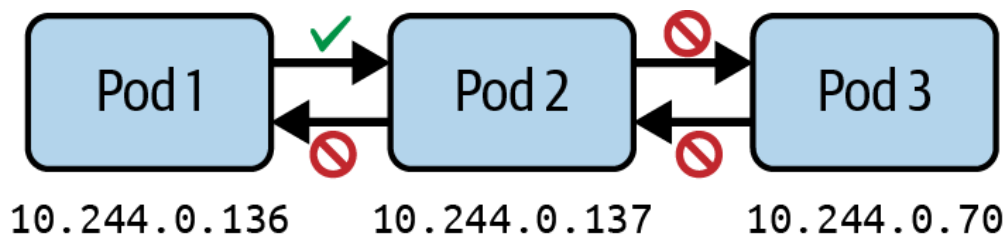


Figure 20-1. Network policies define traffic from and to a Pod

For example, there's no good reason to allow a backend application running in a Pod to talk directly to the frontend application running in another Pod. The communication should be directed from the frontend Pod to the backend Pod.

Installing a Network Policy Controller

A network policy cannot work without a network policy controller. The network policy controller evaluates the collection of rules defined by a network policy. You can find instructions for a wide range of network policy controllers in the [Kubernetes documentation](#). Some CNIs like flannel do not include a network policy controller and focus solely on providing basic Pod-to-Pod connectivity, meaning NetworkPolicy resources will be accepted by the API but completely unenforced.

[Cilium](#) is a CNI that implements a network policy controller. You can install Cilium on cloud-provider and on-prem Kubernetes clusters. Refer to the [installation instructions](#) for detailed information. Once it is installed, you should find at least two Pods running Cilium and the Cilium Operator in the `kube-system` namespace:

```
$ kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
cilium-k5td6	1/1	Running	0	110s
cilium-operator-f5dcdcc8d-njfbk	1/1	Running	0	110s

You can now assume that the rules defined by network policy objects will be evaluated. Additionally, you can use the Cilium command-line tool to validate the proper installation.

Creating a Network Policy

Label selection plays a crucial role in defining which Pods a network policy applies to. We've already seen this concept in action in other contexts (e.g., the [Deployment](#) and the [Service](#)). Furthermore, a network policy defines the direction of the traffic to allow or disallow. In the context of a network policy, incoming traffic is called *ingress*, and outgoing traffic is

called *egress*. For ingress and egress, you can allowlist the sources of traffic like Pods, IP addresses, or ports.

NETWORK POLICIES DO NOT APPLY TO SERVICES

In most cases, you'd set up Service objects to funnel network traffic to Pods based on label and port selection. Network policies do not involve Services at all. All rules are namespace- and Pod-specific.

The creation of network policies is best explained by example. Let's say you're running a Pod that exposes an API to other consumers, for example, a Pod that processes payments for other applications. The company you're working for is migrating applications from a legacy payment processor to a new one. Therefore, you'll want to allow access only from the applications that are capable of properly communicating with it. Right now, you have two consumers—a grocery store and a coffee shop—each running their application in a separate Pod. The coffee shop is ready to consume the API of the payment processor, but the grocery store isn't.

[Figure 20-2](#) shows the Pods and their assigned labels.

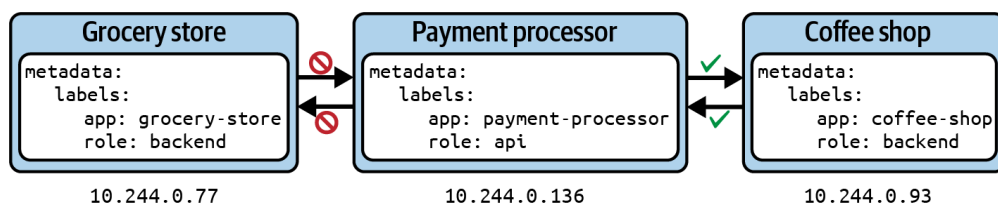


Figure 20-2. Limiting traffic to and from a Pod

Before creating a network policy, we'll stand up the Pods to represent the scenario:

```
$ kubectl run grocery-store --image=nginx:1.25.3-alpine \
  -l app=grocery-store,role=backend --port 80
pod/grocery-store created
$ kubectl run payment-processor --image=nginx:1.25.3-alpine \
  -l app=payment-processor,role=api --port 80
pod/payment-processor created
$ kubectl run coffee-shop --image=nginx:1.25.3-alpine \
  -l app=coffee-shop,role=backend --port 80
```

Given Kubernetes' default behavior of allowing unrestricted Pod-to-Pod communication, the three Pods will be able to talk to one another. The following commands verify the behavior. The grocery store and coffee shop Pods perform a `wget` call to the payment processor Pod's IP address:

```
$ kubectl get pod payment-processor --template '{{.status.podIP}}'
10.244.0.136
$ kubectl exec grocery-store -it -- wget --spider --timeout=1 10.244.0.136
Connecting to 10.244.0.136 (10.244.0.136:80)
remote file exists
$ kubectl exec coffee-shop -it -- wget --spider --timeout=1 10.244.0.136
Connecting to 10.244.0.136 (10.244.0.136:80)
remote file exists
```

You cannot create a new network policy with the imperative `create` command. Instead, you will have to use the declarative approach. The YAML manifest in [Example 20-1](#), stored in the file *networkpolicy-api-allow.yaml*, shows a network policy for the scenario described previously.

Example 20-1. Declaring a NetworkPolicy with YAML

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-allow
spec:
  podSelector: ❶
    matchLabels:
      app: payment-processor
      role: api
  ingress: ❷
  - from:
    - podSelector:
        matchLabels:
          app: coffee-shop
```

❶ Selects the Pod the policy should apply to by label selection

❷ Allows incoming traffic from the Pod with matching labels within the same namespace

A network policy defines a couple of important attributes, which together form its set of rules. [Table 20-1](#) shows the attributes on the `spec` level.

Table 20-1. Spec attributes of a network policy

Attribute	Description
<code>podSelector</code>	Selects the Pods in the namespace to apply the network policy to.
<code>policyTypes</code>	Defines the type of traffic (i.e., ingress and/or egress) the network policy applies to.
<code>ingress</code>	Lists the rules for incoming traffic. Each rule can define <code>from</code> and <code>ports</code> sections.
<code>egress</code>	Lists the rules for outgoing traffic. Each rule can define <code>to</code> and <code>ports</code> sections.

You can specify ingress and egress rules independently using `spec.ingress.from[ ]` and `spec.egress.to[ ]`. Each rule consists of a Pod selector, an optional namespace selector, or a combination of both.

[Table 20-2](#) lists the relevant attributes for the `to` and `from` selectors.

Table 20-2. Attributes of a network policy `to` and `from` selectors

Attribute	Description
<code>podSelector</code>	Selects Pods by label(s) in the same namespace as the network policy that should be allowed as ingress sources or egress destinations
<code>namespaceSelector</code>	Selects namespaces by label(s) for which all Pods should be allowed as ingress sources or egress destinations
<code>namespaceSelector</code> and <code>podSelector</code>	Selects Pods by label(s) within namespaces by label(s)

Let's see the effect of the network policy in action. Create the network policy object from the manifest:

```
$ kubectl apply -f networkpolicy-api-allow.yaml
networkpolicy.networking.k8s.io/api-allow created
```

The network policy prevents calling the payment processor from the grocery store Pod. Accessing the payment processor from the coffee shop Pod works perfectly, as the network policy's Pod selector matches the Pod's assigned label `app=coffee-shop`:

```
$ kubectl exec grocery-store -it -- wget --spider --timeout=1 10.244.0.136
Connecting to 10.244.0.136 (10.244.0.136:80)
wget: download timed out
command terminated with exit code 1
$ kubectl exec coffee-shop -it -- wget --spider --timeout=1 10.244.0.136
Connecting to 10.244.0.136 (10.244.0.136:80)
remote file exists
```

As an administrator, you may be dealing with network policies that have been set up for you by other team members or administrators. You need to know about the `kubectl` commands for listing and inspecting network policy objects to understand their effects on the directional network traffic between microservices.

Listing Network Policies

Listing network policies works the same as any other Kubernetes primitive. Use the `get` command in combination with the resource type `networkpolicy`, or its short-form, `netpol`. For the previous network policy, you see a table that renders the name and Pod selector:

```
$ kubectl get networkpolicy api-allow
```

NAME	POD-SELECTOR	AGE
api-allow	app=payment-processor,role=api	83m

It's unfortunate that the output of the command doesn't give a lot of information about the ingress and egress rules. To retrieve more information, you have to dig into the details.

Rendering Network Policy Details

You can inspect the details of a network policy using the `describe` command. The output renders all the important information: Pod selector, and ingress and egress rules:

```
$ kubectl describe networkpolicy api-allow
```

Name:	api-allow
Namespace:	default
Created on:	2024-01-10 09:06:59 -0700 MST
Labels:	<none>
Annotations:	<none>
Spec:	
PodSelector:	app=payment-processor,role=api
Allowing ingress traffic:	
To Port:	<any> (traffic allowed to all ports)

```
From:
  PodSelector: app=coffee-shop
Not affecting egress traffic
Policy Types: Ingress
```

The network policy details don't draw a clear picture of the Pods that have been selected based on its rules. You can create Pods that match the rules and do not match the rules to verify the network policy's desired behavior.

VISUALIZING NETWORK POLICIES

Defining the rules of network policies correctly can be challenging. The page networkpolicy.io provides a visual editor for network policies that renders a graphical representation in the browser.

As explained earlier, every Pod can talk to other Pods running on any node of the cluster, which exposes a potential security risk. An attacker able to gain access to a Pod theoretically can try to compromise another Pod by communicating with it by its virtual IP address.

Applying Default Network Policies

The principle of least privilege is a fundamental security concept, and it's highly recommended when it comes to restricting Pod-to-Pod network traffic in Kubernetes. The idea is to initially disallow all traffic and then selectively open up only the necessary connections based on the application's architecture and communication requirements.

You can lock down Pod-to-Pod communication with the help of a [default network policy](#). Default network policies are custom policies set up by administrators to enforce restrictive communication patterns by default.

To demonstrate the functionality of such a default network policy, we'll set up two Pods in the namespace `internal-tools`. Within the namespace, all Pods will be able to communicate with each other:

```
$ kubectl create namespace internal-tools
namespace/internal-tools created
$ kubectl run metrics-api --image=nginx:1.25.3-alpine --port=80 \
  -l app=api -n internal-tools
pod/metrics-api created
$ kubectl run metrics-consumer --image=nginx:1.25.3-alpine --port=80 \
```

```
-l app=consumer -n internal-tools  
pod/metrics-consumer created
```

[Example 20-2](#) defines a default network policy that denies all ingress and egress network traffic in the namespace. We'll store the network policy in the file *networkpolicy-deny-all.yaml*.

Example 20-2. Disallowing all traffic with the default policy

```
apiVersion: networking.k8s.io/v1  
kind: NetworkPolicy  
metadata:  
  name: default-deny-all  
  namespace: internal-tools  
spec:  
  podSelector: {}  
  policyTypes:  
    - Ingress  
    - Egress
```

- ❶ The curly braces for `spec.podSelector` mean “apply to all Pods in the namespace.”
- ❷ Defines the types of traffic the rule should apply to, in this case ingress and egress traffic.

Create the network policy from the manifest:

```
$ kubectl apply -f networkpolicy-deny-all.yaml  
networkpolicy.networking.k8s.io/default-deny-all created
```

The network policy prevents any network communication between the Pods in the `internal-tools` namespace, as shown here:

```
$ kubectl get pod metrics-api --template '{{.status.podIP}}' -n internal-  
10.244.0.182  
$ kubectl exec metrics-consumer -it -n internal-tools \  
  -- wget --spider --timeout=1 10.244.0.182  
Connecting to 10.244.0.182 (10.244.0.182:80)  
wget: download timed out  
command terminated with exit code 1  
$ kubectl get pod metrics-consumer --template '{{.status.podIP}}' \  
-n internal-tools  
10.244.0.70  
$ kubectl exec metrics-api -it -n internal-tools \  
  -- wget --spider --timeout=1 10.244.0.70
```



```
Connecting to 10.244.0.70 (10.244.0.70:80)
wget: download timed out
command terminated with exit code 1
```

With those default deny constraints in place, you can define more detailed rules and loosen restrictions gradually. Network policies are additive. It's common practice to now set up additional network policies that will open up directional traffic, but only the ones that are really required.

Restricting Access to Specific Ports

Controlling access at the port level is a critical aspect of network security in Kubernetes. If not explicitly defined by a network policy, all ports are accessible, which can pose security risks. For instance, if you have an application running in a Pod that exposes port 80 to the outside world, leaving all other ports open widens the attack vectors unnecessarily. Port rules can be specified for ingress and egress as part of a network policy. The definition of a network policy in [Example 20-3](#) allows access on port 80.

Example 20-3. Definition of a network policy allowing ingress access on port 8080

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: port-allow
  namespace: internal-tools
spec:
  podSelector:
    matchLabels:
      app: api
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: consumer
    ports:
      - protocol: TCP
        port: 80
```

❶ Only allows incoming traffic on port 80

When defining network policies, only allow those ports that are required for implementing your architectural needs. All other ports should be locked down.

Summary

Inter-Pod communication or communication between two containers of the same Pod is completely unrestricted in Kubernetes. Network policies instate rules to control the network traffic either from or to a Pod. You can think of network policies as firewall rules for Pods. It's best practice to start with a "deny all traffic" rule to minimize the attack vector.

From there, you can open access as needed. Learning about the intricacies of network policies requires a bit of hands-on practice, as it is not directly apparent if the rules work as expected.

Exam Essentials

Understand the purpose and effects of network policies

By default, Pod-to-Pod communication is unrestricted. Instantiate a default deny rule to restrict Pod-to-Pod network traffic with the principle of least privilege. The attribute `spec.podSelector` of a network policy selects the target Pod the rules apply to based on label selection. The ingress and egress rules define Pods, namespaces, IP addresses, and ports for allowing incoming and outgoing traffic.

Know how to implement the principle of least privilege

Network policies can be aggregated. A default deny rule can disallow ingress and/or egress traffic. An additional network policy can open up those rules with a more fine-grained definition.

Explore common network policy scenarios

To explore common scenarios, look at the GitHub repository named ["Kubernetes Network Policy Recipes"](#). The repository comes with a visual representation for each scenario and walks you through the steps to set up the network policy and the involved Pods. This is a great practice resource.

Sample Exercises

Solutions to these exercises are available in [Appendix A](#).

1. Your cluster has two teams working in separate namespaces. You need to implement network policies that control cross-namespace

communication.

Navigate to the directory *app-a/ch20/cross-namespace-control* of the checked-out GitHub repository [bmuschko/cka-study-guide](https://github.com/bmuschko/cka-study-guide). Create the objects from the YAML manifest *setup.yaml*. Inspect the objects in the namespaces `team-alpha` and `team-beta`.

Create a NetworkPolicy in the `team-alpha` namespace that allows the `alpha-app` Pod to connect only to the `team-beta` namespace and denies all other egress traffic except DNS.

Create a NetworkPolicy in the `team-beta` namespace that allows the `beta-app` Pod to receive traffic from `team-alpha` namespace on port 80 and denies all other ingress traffic.

Test the policies to ensure that the `alpha-app` Pod can reach the `beta-app` Pod, the `alpha-app` Pod cannot reach external sites, and the `beta-app` Pod cannot receive traffic from other sources.

2. You have a three-tier application with frontend, backend, and database components. You need to implement network policies to ensure only the `backend` Pod can access the database, while the `frontend` can only communicate with the `backend` Pod.

Navigate to the directory *app-a/ch20/database-access-control* of the checked-out GitHub repository [bmuschko/cka-study-guide](https://github.com/bmuschko/cka-study-guide). Create the objects from the YAML manifest *setup.yaml*. Inspect the objects in the namespace `production`.

Create a NetworkPolicy named `database-policy` that applies to Pods with label `tier=database`. It should only allow ingress from Pods with label `tier=backend` on port 6379.

Create a NetworkPolicy named `backend-policy` that applies to Pods with label `tier=backend`. It should only allow ingress from Pods with label `tier=frontend` on port 80. Allow egress to Pods with label `tier=database` on port 6379. Allow DNS resolution on port 53.

Create a default deny-all ingress policy for the `production` namespace.

Verify your policies by testing connectivity: the `frontend` Pod should *not* be able to reach the database. The `frontend` Pod should be able to reach the `backend` Pod. The `backend` Pod should be able to reach the `database` Pod.